

India AI Ad Creator

Document type: Technical Specification — MVP

Version: 1.0

Stack: Next.js · Supabase · Groq · HuggingFace · Cloudflare R2 · Razorpay

Languages: English · Hindi · Tamil · Telugu

Date: 28 April 2026

This document is the complete technical specification for the India AI Ad Creator MVP. It covers every component of the system end-to-end: database schema, API routes, third-party integrations, data flows, environment configuration, and deployment. It is written for the primary engineer building the product and assumes proficiency with Next.js, Supabase, and PostgreSQL.

Table of Contents

1. Product Overview

- Core value proposition
- Supported languages
- User personas
- Core user flow

2. Finalized Tech Stack

- Decision rationale
- Service summary

3. System Architecture

- Component diagram
- Request lifecycle
- Async vs sync calls

4. Repository Structure

- Directory layout
- Key packages

5. Database Schema

- All tables
- Complete SQL DDL
- Row Level Security policies

6. Supabase Configuration

- Auth setup
- Storage buckets
- Realtime channels

7. Cloudflare R2 Setup

- Bucket structure
- CORS config
- CDN and public access

8. File Upload Pipeline

- Presigned URL flow
- Client-side upload
- Post-upload DB update

9. Background Removal — HuggingFace

- Model and API
- Rate limit handling
- Phash caching
- Fallback strategy

10. Ad Copy Generation — Groq API

- Model choice
- Free tier limits
- System prompt design
- Language prompting
- JSON output schema
- Zod validation
- Retry logic

11. Template Engine — Fabric.js

- Template JSON schema

- Font loading and CORS fix
- Rendering pipeline
- Canvas export

12. End-to-End Generation Flow

- Complete step-by-step
- Timing estimates
- Error states

13. All API Routes

- Route inventory
- Request / response shapes
- Auth middleware

14. Razorpay Payment Integration

- Order creation
- Client-side checkout
- Webhook handling
- Subscription management
- Plan gating middleware

15. Cloudflare Free Tier Usage

- R2 storage and CDN
- Turnstile bot protection
- Cache rules
- Workers rate limiting

16. Pricing Plans and Access Control

- Plan definitions
- Usage quota enforcement

17. Environment Variables

- Complete variable reference

18. Vercel Deployment

- Project setup
- Build config
- Function configuration

19. Cost Breakdown

- Per-service costs at launch and at scale

20. Phase 2 Hooks

- WhatsApp bot placeholder
- Video ad placeholder

Section 1

Product Overview

India AI Ad Creator is a web application that lets Indian small businesses and D2C brands generate professional advertisement creatives in under 30 seconds. The user drops in a product photo or URL, selects a language and format, and receives three ready-to-share ad variants — with copy generated natively in their chosen language, product background removed, and composited onto a branded template.

Supported Languages — MVP

English	en — default, Latin script, Noto Sans
Hindi	hi — Devanagari script, Noto Sans Devanagari
Tamil	ta — Tamil script, Noto Sans Tamil
Telugu	te — Telugu script, Noto Sans Telugu

User Personas

- **D2C fashion / beauty brand:** Sells on Instagram and Meesho. Needs Reels-format ads in Hindi. Budget █499–799/mo.
- **Local service SMB:** Coaching centre, clinic, salon. Active on WhatsApp and Facebook. Wants Tamil or Telugu ad copy. Budget █299–499/mo.
- **Micro-agency:** Manages 5–20 MSME clients. Needs bulk generation across languages. Budget █1,999/mo for Agency plan.

Core User Flow

Step	Stage	Detail
1	Sign up	Google OAuth or email/password via Supabase Auth. Free tier activated immediately.
2	Input	User pastes a product URL or uploads a product photo directly.
3	Extract	Next.js API route scrapes URL with Cheerio, or sends photo to Claude/Groq for product info extraction.
4	BG Remove	Server calls HuggingFace Serverless Inference (RMBG-1.4). Returns transparent PNG stored in R2.
5	Copy Gen	Next.js API route calls Groq (llama-3.3-70b-versatile). Returns 3 copy variants in target language as JSON.
6	Preview	Fabric.js renders template in browser. User sees live preview immediately. Can switch templates and copy variants.
7	Download	canvas.toDataURL() exports 1080x1080 PNG. User downloads directly. Generation saved to DB.

8	Billing gate	On each generation, middleware checks usage count against plan limit. Upsell modal if limit hit.
---	--------------	--

Section 2

Finalized Tech Stack

Every service below was chosen to maximise free-tier coverage, minimise operational complexity for a solo builder, and avoid managed-service lock-in where unnecessary. The stack requires zero infrastructure to maintain at MVP scale.

Layer	Service / Tool	Why	Free tier
Frontend + App	Next.js 14 (App Router)	Known stack, server components reduce client JS, API routes handle backend	Vercel Hobby — unlimited
Hosting	Vercel	Zero-config Next.js deploy, automatic HTTPS, edge network in India	100 GB bandwidth / mo
Database	Supabase (PostgreSQL 15)	Managed Postgres, built-in Auth, Realtime, Row Level Security	500 MB DB, 50K MAU
Auth	Supabase Auth	Google OAuth + email/pass, session management, JWT, integrates with RLS	Included
File Storage	Cloudflare R2	S3-compatible, zero egress fees, Cloudflare CDN, free 10 GB	10 GB storage + free egress
LLM / Copy gen	Groq API (llama-3.3-70b)	Fastest inference (800 tok/s), 14,400 req/day free, excellent Hindi/Tamil/Telugu quality	14,400 req/day free
BG Removal	HuggingFace Serverless (RMBG-1.4)	Best-in-class matting model, free serverless inference with rate limits	~300 req/hr free
Template Engine	Fabric.js (client-side)	In-browser canvas, handles Indic text via Google Fonts, no server compute	Free (npm)
Payments	Razorpay	INR billing, UPI, subscriptions, webhooks, mandatory for India	2% + GST per txn
Email	Resend	Best DX for Next.js, React Email templates	100 emails/day free
Analytics	PostHog	Product analytics, session recording, funnel analysis	1M events/mo free
Error monitoring	Sentry	Catch API failures before users report them	5K errors/mo free
Styling	Tailwind CSS + shadcn/ui	Fast UI development, owned components, Radix accessibility	Free
Bot protection	Cloudflare Turnstile	CAPTCHA alternative on signup and generation endpoints	Free (unlimited)

Layer	Service / Tool	Why	Free tier
State (server)	React Query (TanStack)	Handles generation polling, caching, background refetch	Free (npm)
State (client)	Zustand	Lightweight client state for template/generation UI	Free (npm)
Input validation	Zod	Validate all API inputs AND all Groq JSON outputs	Free (npm)

Section 3

System Architecture

The architecture is intentionally a single Next.js application. There is no separate worker process, message queue, or additional server. All AI API calls are made from Next.js API routes (Route Handlers in App Router). Image rendering happens client-side in Fabric.js. This eliminates Railway, Redis, BullMQ, and node-canvas — the primary sources of operational complexity in a more traditional pipeline.

Component Boundaries

Component	Runs where	Responsibilities
Next.js pages + UI	Browser / Vercel Edge	React UI, Fabric.js canvas, file selection, copy display, download trigger
Next.js API routes	Vercel Serverless Functions	Product scraping, HuggingFace proxy, Groq proxy, Razorpay order/webhook, auth middleware, usage gating
Supabase	Supabase Cloud (us-east-1 or ap-south-1)	PostgreSQL data, Auth sessions, JWT verification, Row Level Security
Cloudflare R2	Cloudflare edge	Product image uploads, BG-removed PNG storage, generated ad previews
Cloudflare Turnstile	Cloudflare edge / client widget	Bot protection on /signup and /api/generate/copy endpoints
Groq API	Groq cloud (external)	LLM inference for ad copy. llama-3.3-70b-versatile model.
HuggingFace Inference	HF cloud (external)	RMBG-1.4 background removal. Called server-side to protect HF token.
Razorpay	Razorpay cloud (external)	Payment processing, subscription management, webhook delivery

Request Lifecycle — Single Ad Generation

Note on Vercel function timeouts: Vercel Hobby has a 10-second timeout on serverless functions. The HuggingFace BG removal call can take 3–8 seconds alone. To work within limits: (1) call BG removal and Groq copy generation in parallel using Promise.all() — total wall time drops to ~6–10 seconds. (2) If timeout is hit on Hobby, upgrade to Vercel Pro (\$20/mo) which allows 60-second functions.

1. User submits generation request from browser (fetch POST /api/generate)
2. Next.js API route validates Supabase session JWT and checks usage quota
3. Middleware rejects if plan limit exceeded – returns 403 with upgrade prompt
4. API route calls Cheerio scraper (if URL input) or parses uploaded image URL
5. Promise.all() – simultaneously:
 - a. POST to HuggingFace Serverless (RMBG-1.4) with product image buffer
 - b. POST to Groq API with language-specific system prompt + product info

6. BG removal result: transparent PNG buffer – uploaded to R2, public URL returned
7. Groq result: JSON {headline, subheadline, cta}[3] – Zod validated
8. API route saves generation record to Supabase (status: complete, output data)
9. API returns JSON to browser: {bg_removed_url, copy_variants[3], generation_id}
10. Browser: Fabric.js loads template, composites product PNG, renders copy
11. User sees live preview. canvas.toDataURL() on download click – PNG to user.

Section 4

Repository Structure

Single Next.js application. No monorepo needed at MVP — keep it simple. All backend logic lives in `app/api/` route handlers. Shared types in a `/lib/types.ts` file.

Project directory tree

```
adcreator/
├── app/
│   ├── (auth)/
│   │   ├── login/page.tsx           # Supabase Auth UI
│   │   ├── signup/page.tsx
│   │   └── (dashboard)/
│   │       ├── layout.tsx           # Protected layout – checks session
│   │       ├── page.tsx             # Generation dashboard
│   │       ├── history/page.tsx     # Past generations
│   │       ├── billing/page.tsx     # Razorpay plans
│   │       └── api/
│   │           ├── generate/
│   │           │   ├── extract/route.ts    # Product info extraction
│   │           │   ├── bgremove/route.ts   # HuggingFace proxy
│   │           │   ├── copy/route.ts       # Groq proxy + copy generation
│   │           │   └── upload/
│   │           │       ├── presign/route.ts # R2 presigned URL generation
│   │           │       └── payments/
│   │           │           ├── create-order/route.ts
│   │           │           ├── webhook/route.ts # Razorpay webhook receiver
│   │           │           └── templates/route.ts # List available templates
│   │           └── layout.tsx
│   └── components/
│       ├── canvas/FabricCanvas.tsx    # Main Fabric.js renderer
│       ├── canvas/TemplateSelector.tsx
│       ├── generate/InputForm.tsx      # URL / file upload
│       ├── generate/CopyVariants.tsx   # 3 copy options selector
│       └── billing/PlanModal.tsx
```

```
■■■ lib/
■   ■■■ supabase/client.ts      # Browser Supabase client
■   ■■■ supabase/server.ts     # Server Supabase client (cookies)
■   ■■■ r2/client.ts           # R2 S3 client
■   ■■■ groq/client.ts         # Groq SDK wrapper
■   ■■■ groq/prompts.ts        # Language-specific system prompts
■   ■■■ hf/bgremove.ts         # HuggingFace inference wrapper
■   ■■■ razorpay/client.ts     #
■   ■■■ schemas/generation.ts  # Zod schemas for all API I/O
■   ■■■ templates/registry.ts  # Template JSON configs + metadata
■   ■■■ types.ts               # Shared TypeScript types
■■■ supabase/
■   ■■■ migrations/           # SQL migration files
■   ■■■ seed.sql              # Template seed data
■■■ public/fonts/             # Noto font files (self-hosted fallback)
■■■ middleware.ts             # Supabase session refresh + route protection
■■■ next.config.mjs
■■■ .env.local                # Never commit this
■■■ package.json
```

Section 5

Database Schema

All tables are in the public schema on Supabase PostgreSQL. The auth.users table is managed by Supabase Auth — we reference it via FK but never write to it. Run all DDL via Supabase migrations (supabase/migrations/*.sql), never via the dashboard.

profiles

One row per authenticated user. Created automatically via Supabase trigger on auth.users insert.

supabase/migrations/001_profiles.sql

```
create table public.profiles (  
  id          uuid primary key references auth.users(id) on delete cascade,  
  full_name   text,  
  avatar_url  text,  
  plan        text not null default 'free'  
              check (plan in ('free', 'starter', 'pro', 'agency')),  
  generation_count int not null default 0,  
  monthly_reset_at timestampz not null default now(),  
  razorpay_customer_id text unique,  
  created_at  timestampz not null default now(),  
  updated_at  timestampz not null default now()  
);  
  
-- Auto-create profile on signup  
create or replace function handle_new_user()  
returns trigger language plpgsql security definer set search_path = public as $$  
begin  
  insert into public.profiles (id, full_name, avatar_url)  
  values (new.id, new.raw_user_meta_data->>'full_name',  
         new.raw_user_meta_data->>'avatar_url');  
  return new;  
end;  
$$;  
create trigger on_auth_user_created  
  after insert on auth.users  
  for each row execute procedure handle_new_user();
```

subscriptions

Tracks active Razorpay subscription per user. One active row per user.

supabase/migrations/002_subscriptions.sql

```
create table public.subscriptions (  
  id                uuid primary key default gen_random_uuid(),  
  user_id           uuid not null references public.profiles(id) on delete cascade,  
  razorpay_subscription_id text unique,  
  razorpay_plan_id   text,  
  plan              text not null check (plan in ('starter', 'pro', 'agency')),  
  status            text not null default 'created'  
                    check (status in ('created', 'active', 'halted', 'cancelled')),  
  current_period_start timestampz,  
  current_period_end timestampz,  
  created_at        timestampz not null default now(),  
  updated_at        timestampz not null default now()  
);  
create index on subscriptions(user_id);  
create index on subscriptions(razorpay_subscription_id);
```

templates

Static template registry. Seeded once. Never user-writable.

```
supabase/migrations/003_templates.sql
```

```
create table public.templates (  
  id          text primary key,          -- e.g. "festival_bright_01"  
  name        text not null,  
  category    text not null,            -- "sale" | "showcase" | "trust" | "urgency"  
  formats     text[] not null,          -- ["1x1", "9x16", "4x5"]  
  preview_url text,                     -- R2 URL of template preview image  
  config      jsonb not null,           -- Fabric.js template JSON  
  is_active   boolean not null default true,  
  created_at  timestampz not null default now()  
);
```

generations

One row per ad generation attempt. Each row stores inputs, outputs, and status.

```
supabase/migrations/004_generations.sql
```

```

create table public.generations (
  id                uuid primary key default gen_random_uuid(),
  user_id           uuid not null references public.profiles(id) on delete cascade,
  -- inputs
  input_type        text not null check (input_type in ('url', 'photo')),
  input_url          text,                -- original URL if input_type=url
  product_image_url text,                -- R2 URL of original product image
  bg_removed_url     text,                -- R2 URL of transparent PNG
  product_name       text,
  product_desc       text,
  language           text not null check (language in ('en', 'hi', 'ta', 'te')),
  template_id        text references public.templates(id),
  format             text not null check (format in ('1x1', '9x16', '4x5')),
  -- outputs
  copy_variants      jsonb,              -- [{headline, subheadline, cta}] x3
  selected_variant   int default 0,      -- 0, 1, or 2
  status             text not null default 'pending'
                    check (status in ('pending', 'processing', 'complete', 'failed')),
  error_message       text,
  created_at         timestampz not null default now()
);
create index on generations(user_id, created_at desc);
create index on generations(status);

```

copy_cache

Semantic copy cache. Keyed by (category, language, tone) to avoid duplicate Groq calls for similar products.

supabase/migrations/005_copy_cache.sql

```

create table public.copy_cache (
  id                uuid primary key default gen_random_uuid(),
  category          text not null,      -- "fashion" | "food" | "electronics" | "beauty" | "home"
  language          text not null check (language in ('en', 'hi', 'ta', 'te')),
  tone              text not null,      -- "festive" | "urgent" | "trust" | "showcase"
  variants          jsonb not null,     -- [{headline, subheadline, cta}] x5 pre-generated
  use_count         int not null default 0,
  created_at        timestampz not null default now()
);
create index on copy_cache(category, language, tone);

-- Increment use_count on cache hit
create or replace function increment_copy_cache_use(cache_id uuid)
returns void language sql as $$
  update copy_cache set use_count = use_count + 1 where id = cache_id;
$$;

```

Row Level Security Policies

RLS policies

```
-- profiles: users can only read/update their own row
alter table profiles enable row level security;
create policy "Users read own profile" on profiles
  for select using (auth.uid() = id);
create policy "Users update own profile" on profiles
  for update using (auth.uid() = id);

-- subscriptions: users read their own only
alter table subscriptions enable row level security;
create policy "Users read own subscription" on subscriptions
  for select using (auth.uid() = user_id);

-- templates: public read, no writes from client
alter table templates enable row level security;
create policy "Public read templates" on templates
  for select using (true);

-- generations: users read/insert their own only
alter table generations enable row level security;
create policy "Users manage own generations" on generations
  for all using (auth.uid() = user_id);

-- copy_cache: server-only (use service_role key from API routes)
alter table copy_cache enable row level security;
-- No client policies – only accessible via service_role in API routes
```

Section 6

Supabase Configuration

Auth Setup

- Enable Google OAuth: Supabase dashboard → Auth → Providers → Google. Add Google Cloud OAuth 2.0 credentials. Set redirect URL to <https://your-project.supabase.co/auth/v1/callback>.
- Enable Email: on by default. Disable email confirmation for MVP (Auth → Email → disable "Enable email confirmations") to reduce signup friction.
- Site URL: set to your Vercel production domain (<https://adcreator.in>). Add localhost:3000 to Additional Redirect URLs for local dev.

Two Supabase Clients

Next.js App Router requires two separate clients — one for browser, one for server (reads cookies for auth state).

lib/supabase/client.ts + server.ts

```
// lib/supabase/client.ts – browser client (use in Client Components)
import { createBrowserClient } from '@supabase/ssr'
export const createClient = () => createBrowserClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
)

// lib/supabase/server.ts – server client (use in API routes, Server Components)
import { createServerClient } from '@supabase/ssr'
import { cookies } from 'next/headers'
export const createServerSupabaseClient = () => {
  const cookieStore = cookies()
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    { cookies: { get: (n) => cookieStore.get(n)?.value } }
  )
}

// For admin operations (bypass RLS) – NEVER expose to client
// Use only in API routes with process.env.SUPABASE_SERVICE_ROLE_KEY
export const createAdminClient = () => createServerClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!,
  { cookies: { get: () => undefined } }
)
```

Middleware — Route Protection

middleware.ts

```
// middleware.ts – runs on every request
import { createServerClient } from '@supabase/ssr'
import { NextResponse } from 'next/server'

export async function middleware(request) {
  const response = NextResponse.next()
  const supabase = createServerClient(URL, ANON_KEY, {
    cookies: {
      get: (n) => request.cookies.get(n)?.value,
      set: (n, v, o) => response.cookies.set({ name: n, value: v, ...o }),
      remove: (n, o) => response.cookies.set({ name: n, value: '', ...o }),
    }
  })
  const { data: { session } } = await supabase.auth.getSession()
  // Redirect unauthenticated users away from dashboard
  if (!session && request.nextUrl.pathname.startsWith('/dashboard')) {
    return NextResponse.redirect(new URL('/login', request.url))
  }
  return response
}
export const config = { matcher: ['/dashboard/:path*', '/api/generate/:path*'] }
```

Section 7

Cloudflare R2 Setup

R2 is an S3-compatible object store with zero egress fees. This is critical for an image-heavy product — serving ad images from S3 or Supabase Storage would incur egress charges at scale. R2 removes that cost entirely.

Bucket Structure

adcreator-uploads	Raw product images uploaded by users. Private. Accessed via presigned URLs only.
adcreator-processed	Background-removed transparent PNGs. Private. Accessed via presigned read URLs.
adcreator-public	Template preview images, watermark assets, static brand assets. Public read via CDN domain.

R2 Client Setup (AWS SDK v3)

lib/r2/client.ts

```
// lib/r2/client.ts
import { S3Client } from '@aws-sdk/client-s3'

export const r2 = new S3Client({
  region: 'auto',
  endpoint: `https://${process.env.CLOUDFLARE_ACCOUNT_ID}.r2.cloudflarestorage.com`,
  credentials: {
    accessKeyId: process.env.R2_ACCESS_KEY_ID!,
    secretAccessKey: process.env.R2_SECRET_ACCESS_KEY!,
  },
})

// Helper: upload buffer to R2
export async function uploadToR2(buffer: Buffer, key: string, contentType: string) {
  const { PutObjectCommand } = await import('@aws-sdk/client-s3')
  await r2.send(new PutObjectCommand({
    Bucket: 'adcreator-processed',
    Key: key,
    Body: buffer,
    ContentType: contentType,
  }))
  return `https://assets.adcreator.in/${key}` // Custom domain on R2 bucket
}
```

Cloudflare CDN on R2

In R2 dashboard: connect the adcreator-public bucket to a custom domain (e.g. assets.adcreator.in). This routes through Cloudflare CDN automatically — images are cached at Cloudflare PoPs globally. Add a Cache Rule in Cloudflare: match URL path /assets/* → Cache Everything, edge TTL 7 days. Generated ad

images served from Indian Cloudflare PoPs (Mumbai, Chennai) will load in <100ms.

R2 CORS Configuration

R2 CORS policy (set via Cloudflare dashboard or wrangler):

```
[
  {
    "AllowedOrigins": ["https://adcreator.in", "http://localhost:3000"],
    "AllowedMethods": ["GET", "PUT", "HEAD"],
    "AllowedHeaders": ["*"],
    "ExposeHeaders": ["ETag"],
    "MaxAgeSeconds": 3600
  }
]
```

Section 8

File Upload Pipeline

Never upload files through your Next.js server — it buffers the entire file in the serverless function memory and counts against your function execution time. Use presigned URLs: the client uploads directly to R2, your API route only generates the signed permission.

Presigned URL Upload Flow

1. Client selects file → sends POST /api/upload/presign with { filename, contentType, size }
2. API route: validate session, check file size (<10MB), check content type (image/*)
3. API route: generate a unique key: uploads/{userId}/{nanoid()}.{ext}
4. API route: call getSignedUrl(r2, new PutObjectCommand(...), { expiresIn: 300 })
5. Return { presignedUrl, key, publicUrl } to client
6. Client: fetch(presignedUrl, { method: "PUT", body: file, headers: { "Content-Type": contentType } })
7. On 200 from R2: client sends POST /api/generate/extract with { imageKey }
8. API route updates generation record with product_image_url = publicUrl

app/api/upload/presign/route.ts

```
// app/api/upload/presign/route.ts
import { getSignedUrl } from '@aws-sdk/s3-request-presigner'
import { PutObjectCommand } from '@aws-sdk/client-s3'
import { r2 } from '@lib/r2/client'
import { createServerSupabaseClient } from '@lib/supabase/server'

export async function POST(req: Request) {
  const supabase = createServerSupabaseClient()
  const { data: { session } } = await supabase.auth.getSession()
  if (!session) return Response.json({ error: 'Unauthorized' }, { status: 401 })

  const { filename, contentType, size } = await req.json()
  if (size > 10 * 1024 * 1024) return Response.json({ error: 'File too large' }, { status: 400 })
  if (!contentType.startsWith('image/')) return Response.json({ error: 'Images only' }, { status: 400 })

  const ext = filename.split('.').pop()
  const key = `uploads/${session.user.id}/${crypto.randomUUID()}.${ext}`

  const presignedUrl = await getSignedUrl(r2, new PutObjectCommand({
    Bucket: 'adcreator-uploads', Key: key, ContentType: contentType
  }), { expiresIn: 300 })

  return Response.json({
    presignedUrl,
    key,
    publicUrl: `${process.env.R2_PUBLIC_BASE}/${key}`
  })
}
```

Section 9

Background Removal — HuggingFace Serverless

Model: briaai/RMBG-1.4. This is a state-of-the-art background matting model. The HuggingFace Serverless Inference API allows calling it without managing GPU infrastructure. On the free tier, rate limits are approximately 300 requests/hour. The API call takes 2–6 seconds per image.

API Call

lib/hf/bgremove.ts

```
// lib/hf/bgremove.ts
import { HfInference } from '@huggingface/inference'

const hf = new HfInference(process.env.HUGGINGFACE_TOKEN)

export async function removeBackground(imageBuffer: Buffer): Promise<Buffer> {
  const blob = new Blob([imageBuffer], { type: 'image/png' })

  // Returns a Blob of the segmented (masked) image
  const result = await hf.imageSegmentation({
    model: 'briaai/RMBG-1.4',
    data: blob,
  })

  // result is an array of segments; the first is the foreground mask
  // Convert the mask to a transparent PNG using sharp
  const maskBuffer = Buffer.from(await result[0].mask.arrayBuffer())
  return applyMaskToImage(imageBuffer, maskBuffer)
}

// Alternative: use the direct HTTP API if SDK has issues
export async function removeBackgroundHTTP(imageBuffer: Buffer): Promise<Buffer> {
  const response = await fetch(
    'https://api-inference.huggingface.co/models/briaai/RMBG-1.4',
    {
      method: 'POST',
      headers: { Authorization: `Bearer ${process.env.HUGGINGFACE_TOKEN}` },
      body: imageBuffer,
    }
  )
  if (!response.ok) throw new Error(`HF error: ${response.status}`)
  return Buffer.from(await response.arrayBuffer())
}
```

Rate Limit Handling and Retry

app/api/generate/bgremove/route.ts

```
// app/api/generate/bgremove/route.ts
export async function POST(req: Request) {
  const { imageUrl } = await req.json()

  // Fetch the image from R2
  const imgRes = await fetch(imageUrl)
  const imageBuffer = Buffer.from(await imgRes.arrayBuffer())

  // Retry up to 3 times with exponential backoff
  let lastError: Error | null = null
  for (let attempt = 0; attempt < 3; attempt++) {
    try {
      const resultBuffer = await removeBackgroundHTTP(imageBuffer)
      // Upload result to R2
      const key = `processed/bgr-${crypto.randomUUID()}.png`
      const url = await uploadToR2(resultBuffer, key, 'image/png')
      return Response.json({ bgRemovedUrl: url })
    } catch (err: any) {
      lastError = err
      if (err.message.includes('429')) {
        // Rate limited – wait before retry
        await new Promise(r => setTimeout(r, 2000 * (attempt + 1)))
      } else if (err.message.includes('503')) {
        // Model loading – wait and retry
        await new Promise(r => setTimeout(r, 5000))
      } else {
        break // Non-retriable error
      }
    }
  }

  return Response.json({ error: lastError?.message }, { status: 500 })
}
```

Perceptual Hash Caching

The same product image uploaded by different users produces an identical BG removal result. Cache by perceptual hash of the input image to avoid duplicate HuggingFace calls. Store the mapping in Supabase (image_phash → r2_url). Check before calling HF.

Perceptual hash cache pattern

```
// Compute pHash of input image before calling HF
import crypto from 'crypto'

// Simple SHA256 of a downscaled image – good enough for deduplication
async function imageHash(buffer: Buffer): Promise<string> {
  // Downscale to 16x16 grayscale for perceptual comparison
  // In practice: use the 'imghash' npm package for true pHash
  return crypto.createHash('sha256').update(buffer).digest('hex').slice(0, 16)
}

// In the bgremove route, before calling HuggingFace:
const hash = await imageHash(imageBuffer)
const { data: cached } = await adminSupabase
  .from('image_cache')
  .select('bg_removed_url')
  .eq('phash', hash)
  .single()

if (cached) return Response.json({ bgRemovedUrl: cached.bg_removed_url })
// else: call HF, then insert hash + result url into image_cache table
```

Section 10

Ad Copy Generation — Groq API

Groq runs llama-3.3-70b-versatile at ~800 tokens/second — an ad copy request completes in under 1 second. The free tier provides 14,400 requests/day and 500,000 tokens/minute. For an MVP with a few hundred users generating a few ads per day, this free tier is more than sufficient.

Groq Client Setup

```
lib/groq/client.ts
```

```
// lib/groq/client.ts
import Groq from 'groq-sdk'
import { z } from 'zod'

export const groq = new Groq({ apiKey: process.env.GROQ_API_KEY })

// Output schema – validated on every response
export const CopyVariantSchema = z.object({
  headline: z.string().max(60), // max 8 words in target language
  subheadline: z.string().max(120), // max 15 words
  cta: z.string().max(30), // max 5 words
})
export const CopyOutputSchema = z.object({
  variants: z.array(CopyVariantSchema).length(3),
})
export type CopyOutput = z.infer<typeof CopyOutputSchema>
```

Language-Specific System Prompts

The system prompt is the most important engineering asset in this product. Each language has a distinct prompt that encodes cultural tone, script constraints, and character limits. Do not use a single generic prompt with a translation instruction — the output quality difference is significant.

```
lib/groq/prompts.ts
```

```
// lib/groq/prompts.ts
export const SYSTEM_PROMPTS: Record<string, string> = {

  en: `You are an expert Indian digital marketing copywriter.
You write high-converting ad copy for Indian small businesses.
Always write in clear, punchy English. Use power words.
Avoid jargon. Keep it benefit-focused. Mention value/price where possible.`,

  hi: `Aap ek expert Indian digital marketing copywriter hain.
Aap Hindi mein high-converting ad copy likhte hain.
Pure Devanagari script mein likhein – Roman Hindi nahi.
Tone: confident, warm, aspirational. Wordplay aur rhyme welcome hai.
Indian values – family, savings, quality – ko reflect karein.`,

  ta: `Neenga oru expert Tamil digital marketing copywriter.
Tamil script-il mattum ezhuthunga – Tanglish venam.
Tone: strong, aspirational, community-focused.
Tamil cultural values – pride, trust, family – reflect pannunga.
CTA-il urgency venum.`,

  te: `Mee oka expert Telugu digital marketing copywriter.
Telugu script lo matrame raayandi – English kalipakandi.
Tone: warm, trustworthy, family-oriented.
Telugu values – quality, relationships, savings – reflect cheyyandi.
CTA clear ga, urgent ga undali.`,
}
```

Copy Generation API Route

```
app/api/generate/copy/route.ts
```

```
// app/api/generate/copy/route.ts
import { groq, CopyOutputSchema } from '@lib/groq/client'
import { SYSTEM_PROMPTS } from '@lib/groq/prompts'
import { createServerSupabaseClient } from '@lib/supabase/server'

export async function POST(req: Request) {
  const supabase = createServerSupabaseClient()
  const { data: { session } } = await supabase.auth.getSession()
  if (!session) return Response.json({ error: 'Unauthorized' }, { status: 401 })

  // Check usage quota
  const profile = await getProfile(session.user.id)
  if (!hasQuotaRemaining(profile)) {
    return Response.json({ error: 'limit_exceeded' }, { status: 403 })
  }

  const { productName, productDesc, language, tone, format } = await req.json()

  // Check copy_cache first
  const cached = await checkCopyCache(productDesc, language, tone)
  if (cached) {
    await incrementGenerationCount(session.user.id)
    return Response.json({ variants: pickRandomVariants(cached.variants) })
  }

  const userPrompt = [
    `Product: ${productName}`,
    `Description: ${productDesc}`,
```

```

`Ad format: ${format} (${format === '9x16' ? 'Reels/Story' : 'Post'})`,
`Tone: ${tone}`,
``,
`Generate exactly 3 ad copy variants. Return ONLY valid JSON:`,
`{"variants": [{"headline": "...", "subheadline": "...", "cta": "..."},`,
`      {"headline": "...", "subheadline": "...", "cta": "..."},`,
`      {"headline": "...", "subheadline": "...", "cta": "..."}]}`,
`No extra text, no markdown, no explanation.`,
].join('\n')

// Retry loop for JSON parse failures
for (let attempt = 0; attempt < 3; attempt++) {
  const completion = await groq.chat.completions.create({
    model: 'llama-3.3-70b-versatile',
    messages: [
      { role: 'system', content: SYSTEM_PROMPTS[language] },
      { role: 'user', content: userPrompt },
    ],
    temperature: 0.8,
    max_tokens: 600,
    response_format: { type: 'json_object' }, // Groq JSON mode
  })

  try {
    const raw = JSON.parse(completion.choices[0].message.content!)
    const parsed = CopyOutputSchema.parse(raw) // Zod validate
    await incrementGenerationCount(session.user.id)
    return Response.json(parsed)
  } catch {
    // Zod parse failed – retry with stricter prompt
    if (attempt === 2) return Response.json({ error: 'copy_failed' }, { status: 500 })
  }
}
}

```

Section 11

Template Engine — Fabric.js Client-Side

The entire image compositing pipeline runs in the browser using Fabric.js. No server-side image generation. This eliminates Sharp, node-canvas, Railway, and all associated complexity. The user sees a live preview that updates instantly when they switch templates or copy variants. Download is a one-liner.

Template JSON Schema

Each template is a JSON config stored in `/lib/templates/` and seeded into the Supabase templates table. The Fabric.js renderer reads this config to draw the canvas.

`lib/templates/types.ts + example config`

```
// lib/templates/types.ts
export interface TemplateConfig {
  id: string
  canvas: { width: number; height: number; background: string }
  layers: Layer[]
}

export type Layer =
  | { type: 'rect'; x: number; y: number; w: number; h: number; fill: string; rx?: number }
  | { type: 'product'; x: number; y: number; w: number; h: number; shadow?: boolean }
  | { type: 'text'; key: 'headline' | 'subheadline' | 'cta'; x: number; y: number;
    maxWidth: number; fontSize: number; fontFamily: string;
    fill: string; fontWeight?: string; textAlign?: string }
  | { type: 'cta_btn'; x: number; y: number; w: number; h: number;
    fill: string; textFill: string; rx: number }
  | { type: 'logo'; x: number; y: number; w: number; h: number }

// Example template: festival_bright_01 (1080x1080)
export const FESTIVAL_BRIGHT: TemplateConfig = {
  id: 'festival_bright_01',
  canvas: { width: 1080, height: 1080, background: '#FF6B35' },
  layers: [
    { type: 'rect', x: 0, y: 800, w: 1080, h: 280, fill: '#111111' },
    { type: 'product', x: 60, y: 80, w: 520, h: 700, shadow: true },
    { type: 'text', key: 'headline', x: 620, y: 120, maxWidth: 420,
      fontSize: 52, fontFamily: 'Noto Sans', fill: 'ffffff' },
    { type: 'text', key: 'subheadline', x: 620, y: 320, maxWidth: 420,
      fontSize: 28, fontFamily: 'Noto Sans', fill: 'eeeeee' },
    { type: 'cta_btn', x: 620, y: 820, w: 320, h: 72, fill: '#FF6B35', textFill: 'fff', rx: 8 },
  ]
}
```

Font Loading — Critical CORS Fix

IMPORTANT: Loading Google Fonts directly into a Fabric.js canvas via CDN URLs will trigger a CORS error when calling `canvas.toDataURL()`. The browser taints the canvas when cross-origin images/fonts are drawn. Fix: proxy the font files through a Next.js API route (`/api/fonts/[family]/route.ts`) that fetches from Google Fonts and returns with correct CORS headers. Fabric.js then loads fonts from your own domain.

Font proxy API routes (critical – eliminates canvas CORS taint)

```
// app/api/fonts/[family]/route.ts – font proxy
export async function GET(req: Request, { params }: { params: { family: string } }) {
  const familyMap: Record<string, string> = {
    'noto-sans': 'Noto+Sans:wght@400;700',
    'noto-sans-devanagari': 'Noto+Sans+Devanagari:wght@400;700',
    'noto-sans-tamil': 'Noto+Sans+Tamil:wght@400;700',
    'noto-sans-telugu': 'Noto+Sans+Telugu:wght@400;700',
  }
  const googleFamily = familyMap[params.family]
  if (!googleFamily) return new Response('Not found', { status: 404 })

  // Fetch the CSS from Google Fonts with a desktop user-agent
  const cssUrl = `https://fonts.googleapis.com/css2?family=${googleFamily}&display=swap`
  const cssRes = await fetch(cssUrl, {
    headers: { 'User-Agent': 'Mozilla/5.0 (compatible)' }
  })
  const css = await cssRes.text()

  // Replace all font file URLs with proxied versions through /api/fontfile/
  const proxied = css.replace(
    /url\(https:\/\/fonts.gstatic.com\/([^\)]+)\)/g,
    (_, path) => `url(/api/fontfile/${encodeURIComponent(path)})`
  )
  return new Response(proxied, { headers: { 'Content-Type': 'text/css', 'Access-Control-Allow-Origin': '*' } })
}

// app/api/fontfile/[...path]/route.ts – proxy actual .woff2 files
export async function GET(req: Request, { params }: { params: { path: string[] } }) {

  const path = params.path.join('/')
  const res = await fetch(`https://fonts.gstatic.com/${encodeURIComponent(path)}`)
  const buf = await res.arrayBuffer()
  return new Response(buf, {
    headers: { 'Content-Type': 'font/woff2', 'Access-Control-Allow-Origin': '*',
      'Cache-Control': 'public, max-age=31536000' }
  })
}
```

Fabric.js Rendering Pipeline

`components/canvas/FabricCanvas.tsx`

```
// components/canvas/FabricCanvas.tsx (simplified)
"use client"
import { useEffect, useRef } from 'react'
import { fabric } from 'fabric'

export function FabricCanvas({ template, productImageUrl, copyVariant, language }) {
  const canvasRef = useRef<HTMLCanvasElement>(null)
  const fabricRef = useRef<fabric.Canvas | null>(null)

  useEffect(() => {
    if (!canvasRef.current) return
    fabricRef.current = new fabric.Canvas(canvasRef.current, {
      width: template.canvas.width,
      height: template.canvas.height,
      selection: false,
    })
    return () => fabricRef.current?.dispose()
  }, [])

  useEffect(() => {
    if (!fabricRef.current || !template || !copyVariant) return
    renderTemplate(fabricRef.current, template, productImageUrl, copyVariant, language)
  }, [template, productImageUrl, copyVariant, language])

  const handleDownload = () => {
    const dataUrl = fabricRef.current?.toDataURL({
      format: 'png', multiplier: 1, quality: 1
    })
  }
}
```

```

    const a = document.createElement('a')
    a.href = dataUrl!
    a.download = `ad-${Date.now()}.png`
    a.click()
  }

  return <canvas ref={canvasRef} />
}

async function renderTemplate(canvas, template, imgUrl, copy, language) {
  canvas.clear()
  canvas.setBackgroundColor(template.canvas.background, canvas.renderAll.bind(canvas))

  // Load font first
  const fontFamily = FONT_MAP[language]
  await loadFont(fontFamily) // fetches from /api/fonts/[family]

  for (const layer of template.layers) {
    if (layer.type === 'rect') {
      canvas.add(new fabric.Rect({ left: layer.x, top: layer.y,
        width: layer.w, height: layer.h, fill: layer.fill, selectable: false }))
    }
    if (layer.type === 'product') {
      const img = await loadFabricImage(imgUrl)
      img.set({ left: layer.x, top: layer.y, selectable: false })
      img.scaleToFit(layer.w, layer.h)
      canvas.add(img)
    }
  }

  if (layer.type === 'text') {
    const text = copy[layer.key]
    canvas.add(new fabric.Textbox(text, {
      left: layer.x, top: layer.y, width: layer.maxWidth,
      fontSize: layer.fontSize, fontFamily: `${fontFamily}, sans-serif`,
      fill: layer.fill, selectable: false, splitByGrapheme: true,
    }))
  }
}
canvas.renderAll()
}

```

Section 12

End-to-End Generation Flow

Complete sequence from user action to downloadable ad image, with timing estimates and error handling at each step.

S t e p	Action	Where	Time	Error handling
1	User uploads photo or pastes URL	Browser	—	File size check client-side before upload
2	Fetch presigned URL	POST /api/upload/presign	50ms	Session expired → redirect to login
3	Upload file direct to R2	Browser → R2	500ms –2s	Retry once on network fail
4	Extract product info (URL: Cheerio / Photo: send URL to Groq Vision)	POST /api/generate/extract	500ms –2s	Cheerio fail → prompt user to fill manually
5	Remove background (HuggingFace RMBG-1.4)	POST /api/generate/bgremove	3–8s	Retry 3× with backoff; 503 = model loading, wait 5s
6	Generate copy (Groq llama-3.3-70b)	POST /api/generate/copy	500ms –1.5s	Zod parse fail → retry prompt up to 3×; quota exceeded → 403
7	Save generation record to Supabase	Server (inside step 5/6 route)	100ms	Non-critical; log error, do not block response
8	Return results to browser	HTTP response	—	Any 5xx → show retry button; 403 → show upgrade modal
9	Fabric.js renders canvas (template + product + copy)	Browser	200–500ms	Font load fail → fallback to system font, retry font load
10	User clicks Download → canvas.toDataURL()	Browser	<100ms	Canvas tainted error → font CORS issue (see Section 11)

Parallel Calls — Performance Optimization

Steps 5 and 6 (BG removal + copy generation) are independent. Run them in parallel with Promise.all() to halve total wait time.

Parallel execution pattern

```
// In the main generation orchestrator route or client-side:
const [bgResult, copyResult] = await Promise.all([
  fetch('/api/generate/bgremove', {
    method: 'POST', body: JSON.stringify({ imageUrl: productImageUrl })
  }).then(r => r.json()),

  fetch('/api/generate/copy', {
    method: 'POST',
    body: JSON.stringify({ productName, productDesc, language, tone, format })
  }).then(r => r.json()),
])

// Total wall time: max(bgRemovalTime, copyGenTime) ≈ 4-8 seconds
// vs sequential: bgRemovalTime + copyGenTime ≈ 8-16 seconds
```

Section 13

All API Routes

Route	Method	Auth?	Purpose
/api/upload/presign	POST	Yes	Generate R2 presigned PUT URL for direct client upload
/api/generate/extract	POST	Yes	Scrape product info from URL (Cheerio) or extract from image (Groq Vision)
/api/generate/bgremove	POST	Yes	Proxy call to HuggingFace RMBG-1.4. Returns R2 URL of transparent PNG.
/api/generate/copy	POST	Yes	Proxy call to Groq. Validates output with Zod. Checks/updates usage quota.
/api/templates	GET	No	Return list of active templates from Supabase templates table.
/api/generations	GET	Yes	Return paginated list of user's past generations.
/api/generations/[id]	GET	Yes	Return single generation by ID (ownership checked via RLS).
/api/payments/create-order	POST	Yes	Create Razorpay order or subscription. Returns order_id for checkout.
/api/payments/webhook	POST	No (sig verify)	Receive Razorpay webhook events. Verify HMAC. Update subscription status.
/api/fonts/[family]	GET	No	Proxy Google Fonts CSS with CORS headers for canvas use.
/api/fontfile/[...path]	GET	No	Proxy individual .woff2 font files with CORS headers.
/api/turnstile/verify	POST	No	Verify Cloudflare Turnstile token on sensitive endpoints.

Usage Quota Middleware

Called from within /api/generate/copy before the Groq call. Checks and increments generation count per billing period.

```
lib/quota.ts
```

```
// lib/quota.ts
const PLAN_LIMITS = {
  free: 5,
  starter: 50,
  pro: 200,
  agency: Infinity
}

export async function checkAndIncrementQuota(userId: string): Promise<
  { allowed: boolean; remaining: number }
> {
  const adminSupabase = createAdminClient()
  const { data: profile } = await adminSupabase
    .from('profiles')
    .select('plan, generation_count, monthly_reset_at')
    .eq('id', userId)
    .single()

  // Reset monthly counter if new billing period
  const resetAt = new Date(profile.monthly_reset_at)
  if (Date.now() > resetAt.getTime() + 30 * 24 * 60 * 60 * 1000) {
    await adminSupabase.from('profiles')
      .update({ generation_count: 0, monthly_reset_at: new Date().toISOString() })
      .eq('id', userId)
    profile.generation_count = 0
  }

  const limit = PLAN_LIMITS[profile.plan]

  if (profile.generation_count >= limit) return { allowed: false, remaining: 0 }

  await adminSupabase.from('profiles')
    .update({ generation_count: profile.generation_count + 1 })
    .eq('id', userId)

  return { allowed: true, remaining: limit - profile.generation_count - 1 }
}
```

Section 14

Razorpay Payment Integration

Razorpay handles INR billing, UPI, cards, and subscription management. The integration has three parts: order/subscription creation (server), checkout (client), and webhook handling (server). Razorpay charges 2% + GST per transaction with no monthly platform fee.

Plan Configuration

Free	No Razorpay plan needed. Tracked by profiles.plan = "free" with 5 generations/month.
Starter — █499/mo	Create a Razorpay Plan (monthly interval, █499). Store plan_id in env as RAZORPAY_PLAN_STARTER.
Pro — █1,199/mo	Create a Razorpay Plan (monthly, █1199). Store as RAZORPAY_PLAN_PRO.
Agency — █2,999/mo	Create a Razorpay Plan (monthly, █2999). Store as RAZORPAY_PLAN_AGENCY.
Annual discount	Create matching annual plans at 10× monthly price (2 months free). Separate plan_ids.

Order Creation

app/api/payments/create-order/route.ts

```
// app/api/payments/create-order/route.ts
import Razorpay from 'razorpay'

const razorpay = new Razorpay({
  key_id: process.env.RAZORPAY_KEY_ID!,
  key_secret: process.env.RAZORPAY_KEY_SECRET!,
})

export async function POST(req: Request) {
  const { plan } = await req.json()
  const planIds = {
    starter: process.env.RAZORPAY_PLAN_STARTER,
    pro: process.env.RAZORPAY_PLAN_PRO,
    agency: process.env.RAZORPAY_PLAN_AGENCY,
  }

  const subscription = await razorpay.subscriptions.create({
    plan_id: planIds[plan],
    total_count: 12, // 12 months (or 1 for month-to-month – use 120 for annual)
    quantity: 1,
    customer_notify: 1,
  })

  // Save pending subscription to DB
  await adminSupabase.from('subscriptions').upsert({
    user_id: session.user.id,
    razorpay_subscription_id: subscription.id,
    plan, status: 'created'
  })

  return Response.json({ subscriptionId: subscription.id })
}
```

Webhook Handler — Critical: Always Verify Signature

```
app/api/payments/webhook/route.ts
```

```
// app/api/payments/webhook/route.ts
import crypto from 'crypto'

export async function POST(req: Request) {
  const body = await req.text()
  const signature = req.headers.get('x-razorpay-signature')!

  // ALWAYS verify signature before processing
  const expectedSig = crypto
    .createHmac('sha256', process.env.RAZORPAY_WEBHOOK_SECRET!)
    .update(body)
    .digest('hex')
  if (expectedSig !== signature) {
    return new Response('Invalid signature', { status: 400 })
  }

  const event = JSON.parse(body)

  switch (event.event) {
    case 'subscription.activated':
      await handleSubscriptionActive(event.payload.subscription.entity)
      break
    case 'subscription.charged':
      // Payment successful – update period dates
      await handleSubscriptionCharged(event.payload.subscription.entity)
      break
    case 'subscription.cancelled':
      await handleSubscriptionCancelled(event.payload.subscription.entity)
      break
    case 'subscription.halted':
      // Payment failed – downgrade to free after grace period
      await handleSubscriptionHalted(event.payload.subscription.entity)
      break
  }
  return new Response('ok')
}

async function handleSubscriptionActive(sub: any) {
  const planMap = {
    [process.env.RAZORPAY_PLAN_STARTER!]: 'starter',
    [process.env.RAZORPAY_PLAN_PRO!]: 'pro',
    [process.env.RAZORPAY_PLAN_AGENCY!]: 'agency',
  }
  const plan = planMap[sub.plan_id]
  await adminSupabase.from('subscriptions')
    .update({ status: 'active', current_period_start: new Date(sub.start_at * 1000),
      current_period_end: new Date(sub.end_at * 1000) })
    .eq('razorpay_subscription_id', sub.id)
  await adminSupabase.from('profiles')
    .update({ plan })
    .eq('razorpay_customer_id', sub.customer_id)
}
```

Section 15

Cloudflare Free Tier Usage

Cloudflare offers an extraordinarily generous free tier. Use every applicable feature.

Feature	What we use it for	Free limit
Cloudflare DNS	Manage adcreator.in DNS. All records. Automatic proxying for CDN.	Unlimited
Cloudflare CDN	Proxy all traffic through Cloudflare PoPs. Indian PoPs: Mumbai, Chennai, Delhi. Automatic caching of static assets.	Unlimited bandwidth
R2 Object Storage	Store all product images, BG-removed PNGs, template previews. Zero egress fees.	10 GB / mo free + free egress
R2 Custom Domain	assets.adcreator.in serves R2 objects via Cloudflare CDN. Cache hit = zero origin request.	Free with R2
Cloudflare Turnstile	Bot protection on /signup and /api/generate/copy. Invisible by default. Replace reCAPTCHA.	Unlimited (free forever)
Cache Rules	Cache all /assets/* URLs at edge for 7 days. Generated ad images served from nearest PoP.	Included in free plan
Cloudflare Pages	Not needed (using Vercel), but available as free Vercel alternative if needed.	500 builds/mo
Email Routing	Route support@adcreator.in to personal inbox. Free transactional email routing.	Unlimited addresses

Turnstile Integration

Cloudflare Turnstile integration

```
// 1. Add Turnstile widget to signup and generation pages (client component)
import { Turnstile } from '@marsidev/react-turnstile'

<Turnstile
  siteKey={process.env.NEXT_PUBLIC_TURNSTILE_SITE_KEY}
  onSuccess={({token}) => setTurnstileToken(token)}
/>

// 2. Include token in API request
body: JSON.stringify({ ...generationPayload, turnstileToken })

// 3. Verify in API route before processing
const verifyRes = await fetch('https://challenges.cloudflare.com/turnstile/v0/siteverify', {
  method: 'POST',
  body: JSON.stringify({
    secret: process.env.TURNSTILE_SECRET_KEY,
    response: turnstileToken,
    remoteip: req.headers.get('CF-Connecting-IP'),
  })
})
const { success } = await verifyRes.json()
if (!success) return Response.json({ error: 'Bot detected' }, { status: 403 })
```

Section 16

Pricing Plans and Access Control

Plan	Price	Generations/mo	Languages	Formats	Features
Free	■0	5	English, Hindi	1×1 Post only	Watermark on output. Upgrade prompt at limit.
Starter	■499/mo	50	All 4 (EN/HI/TA/TE)	All 3 formats	No watermark. HD download. All templates.
Pro	■1,199/mo	200	All 4	All 3 formats	All Starter + Brand kit (Phase 2). Priority generation.
Agency	■2,999/mo	Unlimited	All 4	All 3 formats	All Pro + API access (Phase 2). Bulk generation.

Watermark Implementation (Free Tier)

Apply watermark in Fabric.js before canvas.toDataURL() on free tier users.

Watermark for free tier

```
// In FabricCanvas – apply watermark for free users
if (userPlan === 'free') {
  const watermark = new fabric.Text('adcreator.in', {
    left: template.canvas.width - 20,
    top: template.canvas.height - 20,
    fontSize: 18,
    fill: 'rgba(255,255,255,0.6)',
    fontFamily: 'sans-serif',
    originX: 'right',
    originY: 'bottom',
    selectable: false,
    shadow: new fabric.Shadow({ color: 'rgba(0,0,0,0.3)', blur: 4 })
  })
  canvas.add(watermark)
}
```

Section 17

Environment Variables

All variables go in `.env.local` (never committed). Set production values in Vercel dashboard under Settings → Environment Variables.

Variable	Value source	Used in
NEXT_PUBLIC_SUPABASE_URL	Supabase dashboard → Project settings → API	Client + server
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase dashboard → Project settings → API	Client + server
SUPABASE_SERVICE_ROLE_KEY	Supabase dashboard → Project settings → API (keep secret!)	Server only — admin ops, bypass RLS
GROQ_API_KEY	console.groq.com → API Keys	Server only
HUGGINGFACE_TOKEN	huggingface.co → Settings → Access Tokens	Server only
CLOUDFLARE_ACCOUNT_ID	Cloudflare dashboard → right sidebar	Server only
R2_ACCESS_KEY_ID	R2 → Manage R2 API Tokens → Create Token	Server only
R2_SECRET_ACCESS_KEY	R2 → Manage R2 API Tokens → Create Token	Server only
R2_PUBLIC_BASE	https://assets.adcreator.in (your R2 custom domain)	Server only
RAZORPAY_KEY_ID	Razorpay dashboard → Settings → API Keys	Client (for checkout) + server
RAZORPAY_KEY_SECRET	Razorpay dashboard → Settings → API Keys	Server only
RAZORPAY_WEBHOOK_SECRET	Razorpay dashboard → Webhooks → Secret	Server only
RAZORPAY_PLAN_STARTER	Razorpay dashboard → Subscriptions → Plans	Server only
RAZORPAY_PLAN_PRO	Razorpay dashboard → Subscriptions → Plans	Server only
RAZORPAY_PLAN_AGENCY	Razorpay dashboard → Subscriptions → Plans	Server only
NEXT_PUBLIC_TURNSTILE_SITE_KEY	Cloudflare dashboard → Turnstile → Add site	Client (widget)

Variable	Value source	Used in
TURNSTILE_SECRET_KEY	Cloudflare dashboard → Turnstile → Add site	Server only (verify)
RESEND_API_KEY	resend.com → API Keys	Server only
NEXT_PUBLIC_POSTHOG_KEY	posthog.com → Project → API Key	Client
SENTRY_DSN	sentry.io → Project → DSN	Client + server

Section 18

Vercel Deployment

Initial Setup

1. Push repository to GitHub (or GitLab). Keep main branch as production.
2. Import project in Vercel dashboard → Add New Project → select repo.
3. Framework Preset: Next.js (auto-detected). Root directory: / (or /apps/web if monorepo).
4. Add all environment variables from Section 17 in Vercel → Settings → Environment Variables. Set each for Production, Preview, and Development as appropriate.
5. Deploy. Vercel provides a .vercel.app URL for preview and maps your custom domain.
6. Add custom domain (adcreator.in) in Vercel → Settings → Domains. Cloudflare handles DNS — set A/CNAME records as Vercel instructs, but proxy through Cloudflare (orange cloud ON).

next.config.mjs

next.config.mjs

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  images: {
    remotePatterns: [
      { protocol: 'https', hostname: 'assets.adcreator.in' }, // R2 CDN
      { protocol: 'https', hostname: '*.supabase.co' },
    ],
  },
  // Allow larger API route body for image uploads
  // (not needed with presigned URL approach – uploads bypass Next.js)
  experimental: {
    serverComponentsExternalPackages: ['sharp'], // if you add sharp later
  },
}
export default nextConfig
```

Vercel Function Configuration

Hobby plan: 10-second max duration. Pro plan: 60-second max. The HuggingFace BG removal call takes 3–8 seconds. Running BG removal and Groq in parallel (Promise.all) brings total time to ~8 seconds — within Hobby limits on good days, at risk on slow HF cold starts. Recommendation: start on Hobby, upgrade to Pro (\$20/mo) when users report timeouts.

Function timeout config

```
// To increase timeout on specific routes (Vercel Pro required for >10s):  
// Add to the route file:  
export const maxDuration = 60 // seconds – requires Vercel Pro  
  
// Add to app/api/generate/bgremove/route.ts and copy/route.ts
```

Section 19

Cost Breakdown

At Launch (pre-revenue, ~100 test generations/mo)

Service	Cost	Notes
Vercel	■0	Hobby plan, free tier
Supabase	■0	Free tier — 500MB DB, 1GB storage, 50K MAU — plenty for MVP
Groq API	■0	14,400 req/day free. 100 test generations well within limit.
HuggingFace	■0	Free tier — ~300 req/hr. 100/day is fine.
Cloudflare R2	■0	10GB free storage, zero egress. Plenty for thousands of images.
Cloudflare Turnstile	■0	Free forever, unlimited.
Resend (email)	■0	100 emails/day free — covers welcome + billing emails.
PostHog	■0	1M events/mo free.
Sentry	■0	5K errors/mo free.
Domain (.in)	~■800/yr	One-time annual cost. ~■67/mo amortized.
Razorpay	2% + GST	No monthly fee. Only when transactions happen.
TOTAL	~■67/mo	Effectively free until you have paying users.

At 500 Paying Users (avg 40 ads/user/mo = 20,000 generations/mo)

Service	Cost/mo	Calculation
Vercel Pro	■1,660	Needed for 60s function timeouts at this scale
Supabase Pro	■2,080	8GB DB storage, 100GB file storage (R2 replaces Supabase Storage)
Groq API	■0–4,950	Free tier: 14,400 req/day = 432K/mo >> 20K. May hit token limits. Paid: \$0.59/M tokens. 20K gens × 500 tokens = 10M tokens = ~\$5.90 = ■491.
HuggingFace Pro	■745	\$9/mo Pro for unlimited rate — needed at this scale
Cloudflare R2	~■830	~50GB stored × \$0.015 = \$0.75 + operations = ~\$10
Resend	■1,660	Pro plan for volume

Service	Cost/mo	Calculation
TOTAL (infra)	~\$7,000/mo	vs ~\$60,000+ MRR at 500 users on mixed plans
Gross margin	>85%	Even at scale, infra is a small fraction of revenue

Section 20

Phase 2 Hooks — Placeholders in MVP Code

The MVP codebase should include placeholder structures for Phase 2 features so that adding them later does not require architectural changes.

WhatsApp Bot (Phase 2)

Add a whatsapp_sessions table to Supabase now (even if unused). The bot will be a separate Next.js API route (/api/whatsapp/webhook) that reuses the same extract/bgremove/copy route logic.

WhatsApp session table (seed now, use Phase 2)

```
-- Add now, use in Phase 2
create table public.whatsapp_sessions (
  phone_number text primary key,
  user_id      uuid references public.profiles(id),
  step         text default 'idle',  -- idle | awaiting_language | awaiting_format | generating
  language     text,
  format       text,
  input_image_url text,
  last_activity timestampz default now()
);
```

Video Ads (Phase 2 — Creatomate)

Add a video_format column to the templates table now. Set is_video = false for all MVP templates. Phase 2 templates set is_video = true and include a creatomate_template_id.

Video template scaffold

```
-- Add to templates table in a Phase 2 migration
alter table public.templates add column is_video boolean not null default false;
alter table public.templates add column creatomate_template_id text;

-- In the generation flow, check is_video flag:
// if (template.isVideo) {
//   return callCreatomateAPI(template.creatomateTemplateId, payload)
// } else {
//   return renderWithFabric(template, payload) // current MVP path
// }
```

Brand Kit (Phase 2)

Brand kit table (add now, implement Phase 2)

```
-- Add brand kit table now (unused until Phase 2)
create table public.brand_kits (
  id          uuid primary key default gen_random_uuid(),
  user_id     uuid not null references public.profiles(id) on delete cascade,
  name        text not null default 'My Brand',
  logo_url    text,          -- R2 URL of uploaded logo
  primary_color text,        -- hex color
  secondary_color text,
  font_family text default 'Noto Sans',
  created_at  timestamptz not null default now()
);
alter table brand_kits enable row level security;
create policy "Users manage own brand kits" on brand_kits
  for all using (auth.uid() = user_id);
```

End of Technical Specification

India AI Ad Creator · MVP · April 2026 · Stack: Next.js · Supabase · Groq · HuggingFace · Cloudflare R2 · Razorpay